ESCUELA DE INGENIERÍA ELÉCTRICA

PONTIFICIA UNIVERSIDAD CATÓLICA DE VALPARAÍSO

Guía 2

Comunicación Serial y FastApi

Curso: EIE390 - Robótica e Inteligencia Artificial
Profesor: Guillermo Cid Ampuero

Objetivos

- Objetivo 1: Comprender los fundamentos de la comunicación serial y su aplicación en la interacción entre un computador y la Raspberry Pi Pico.
- Objetivo 2: Implementar comunicación serial mediante programación en Python y MicroPython para el envío y recepción de datos.
- Objetivo 3: Desarrollar una interfaz básica utilizando Tkinter que permita enviar comandos a la Raspberry Pi Pico a través del puerto serial.

Repositorio del Curso

1 Introducción

La comunicación serial es un protocolo de comunicación que envía bits de manera secuencial, es decir, los envía uno a uno.

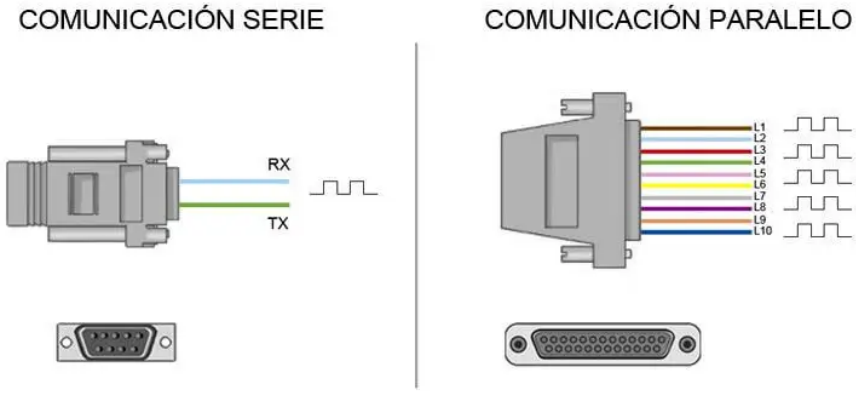

Figura 1: Diferencia entre comunicación Serial y en Paralelo

El concepto de comunicación serial es sencillo. El puerto serial envía y recibe bytes de información un bit a la vez. Aun y cuando esto es más lento que la comunicación en paralelo, que permite la transmisión de un byte completo por vez, este método de comunicación es más sencillo y puede alcanzar mayores distancias. Por ejemplo, la especificación IEEE 488 para la comunicación en paralelo determina que el largo del cable para el equipo **no puede ser mayor a 20 metros**, con no más de 2 metros entre dos dispositivos; por el otro lado, utilizando comunicación serial el largo del cable puede llegar a los **1200 metros**.

En la comunicación serial, los datos son enviados en forma de pulsos binarios. En otras palabras, podemos decir que el binario 1 significa un lógico alto o 5 voltios, y el 0 significa un lógico bajo o 0 voltios.

La comunicación serial puede adoptar varias formas, dependiendo del modo de transmisión y del tipo de transmisión de datos.

1.1 Simplex

Simplex se refiere a la tecnología de comunicación unidireccional. Solo hay un cliente (el emisor o el receptor está activo al mismo tiempo). Si el emisor envía, el receptor solo puede aceptar.

Figura 2: Comunicación Simple

1.2 Half-Duplex

En la comunicación Half-Duplex, ambos pueden enviar, tanto receptor como emisor, pero no a la misma vez, es decir. El transmisor puede enviar, y el receptor recibe, y una vez recibido, puedo enviar y así viceversa.

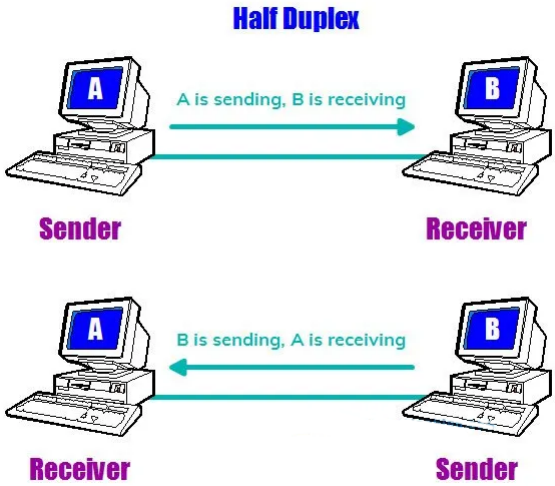

Figura 3: Comunicación Half-Duplex

1.3 Full-Duplex

En la comunicación Full-Duplex, tanto emisor como receptor pueden enviar y recibir a la vez.

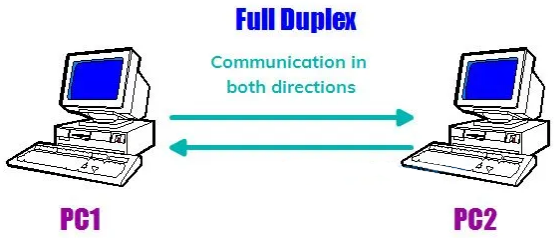

Figura 4: Comunicación Full - Duplex

2 Desarrollo 1

Para comenzar vamos a probar la comunicación serial entre el PC y la RPi Pico, para esto necesitaremos dos codigos:

- Código del Pc, que envia datos a nuestra RPi Pico.
- Código de la RPi Pico, que recibe los datos.

El ejercicio que vamos a realizar consiste en enviar desde el computador hacia la RPI Pico los comando *ON* y *OFF* para encender el LED de la RPI Pico.

2.1 Script en la Raspberry Pi Pico

Como aprendimos en la clase pasada, para ejecutar un programa de manera automática en la Raspberry Pi Pico es necesario crear un archivo llamado `main.py`, dentro del cual se debe colocar el código que deseamos ejecutar. El código que vamos a cargar en esta sección en la RPI Pico se encuentra a continuación:

Codigo_RPI_Pico/ejemplo_1.py

A continuación, revisaremos algunas secciones importantes del código utilizado.

Librería `sys`: La librería `sys` permite controlar el flujo de datos que ingresan y salen de la Raspberry Pi Pico a través de la comunicación serial USB.

- *sys.stdin*: representa los datos que ingresan a la Raspberry Pi Pico desde el computador.
- *sys.stdout*: representa los datos que salen desde la Raspberry Pi Pico hacia el computador (la función `print()` utiliza internamente este canal).

La siguiente línea de código:

```
cmd = sys.stdin.readline().strip().lower()
```

permite recibir y procesar un comando enviado por comunicación serial. Cada función cumple un rol específico:

- `readline()`: lee una línea completa enviada desde el computador.
- `strip()`: elimina espacios en blanco y saltos de línea al inicio y final del texto.
- `lower()`: convierte el texto a minúsculas para facilitar la comparación de comandos.

Para verificar inicialmente que el código cargado en la Raspberry Pi Pico funciona correctamente, enviaremos comandos seriales directamente desde la terminal del computador. De esta manera podremos comprobar si se ejecutan correctamente las acciones programadas, como el encendido y apagado del LED.

2.2 Script de Control en Python (Computador)

Para establecer esta comunicación utilizaremos el lenguaje Python junto con la librería `pyserial`, la cual nos permite interactuar con los puertos USB del equipo de forma sencilla. Para esto necesitamos instalarla mediante:

```
python -m pip install pyserial
```

El código que ejecutaremos en el computador es el siguiente:

Codigo_Python/ejemplo_1.py

A continuación, revisaremos las secciones y métodos más críticos de este script:

Configuración del Puerto Serial: La conexión física se inicializa instanciando la clase `serial.Serial`, a la cual debemos entregarle los parámetros fundamentales de la comunicación:

- *port*: indica el identificador del puerto asignado por el sistema operativo (por ejemplo, `'COM3'` en Windows).
- *baudrate*: establece la velocidad de transmisión en baudios (bits por segundo). Debe ser idéntica a la configurada internamente en la Pico.
- *dsrdtr=False*: Desactiva la señal DTR (*Data Terminal Ready*), evitando que la placa sufra un reinicio no deseado (reset).

Envío y Recepción de Mensajes: Es importante entender que la comunicación por puerto serial se realiza mediante *bytes*, no como texto (*strings*). Las siguientes líneas convierten entre ambos formatos:

```
s.write((cmd + '\r\n').encode())
respuesta = s.readline().decode().strip()
```

- `encode()`: convierte el texto a bytes.
- `'\r\n'`: indica el fin del comando.
  - r (Retorno de Carro): Le dice al sistema que mueva el cursor de vuelta al principio de la línea actual. (Viene de las antiguas máquinas de escribir, cuando el rodillo o çarro"volvía a la derecha).
  - n (Salto de Línea): Le dice al sistema que baje el cursor a la siguiente línea.
- `write()`: envía los bytes por el puerto serial.
- `decode()`: convierte los bytes recibidos a texto.

Finalmente, la instrucción `s.close()` garantiza que el puerto COM sea liberado al terminar el programa. Si se omite este paso, el puerto quedará .ºcupado.en el sistema operativo, impidiendo futuras conexiones o reprogramaciones de la placa.

3 Desarrollo 2

3.1 Comunicación Serial No Bloqueante

En aplicaciones de robótica, el uso de funciones bloqueantes como `readline()` en la RPI Pico representa un problema crítico, ya que detiene la ejecución del procesador hasta que se recibe un carácter de fin de línea. Para evitar esto, implementamos un esquema de **Polling** utilizando la librería `uselect`.

- **Objeto Poll:** Se crea una instancia de `uselect.poll()` que monitorea el flujo de entrada estándar (`stdin`).
- **Método `poll(timeout)`:** Esta función devuelve una lista vacía si no hay datos, permitiendo que el bucle `while` continúe su ejecución. Si hay datos presentes, devuelve el evento para proceder con la lectura.
- **Ventaja en Robótica:** Al no quedar `atrapado` esperando el puerto serial, el controlador puede mantener la frecuencia de muestreo de los sensores y la actualización de los actuadores de forma constante.

El codigo de esta prueba se encuentra en :

- **Código_RPI_Pico/ejemplo_2.py**
- **Código_Python/ejemplo_2.py**

El código de la Pc, puede ser reutilizado del anteriore, o usar el que se entrega en el cuadro verde. Sin embargo, el código de la RPI Pico posee modificaciones que se explicarán a continuación:

3.2 Script No Bloqueante en la Raspberry Pi Pico

Librería `uselect`: En robótica, es crítico que el microcontrolador no se quede congelado esperando instrucciones. La librería `uselect` permite implementar un esquema de sondeo (*polling*) para revisar de forma asíncrona si hay datos entrantes.

Las siguientes líneas preparan este sistema de monitoreo:

```
spoll = uselect.poll()
spoll.register(sys.stdin, uselect.POLLIN)
```

- *uselect.poll()*: crea un objeto de sondeo que permite vigilar múltiples flujos de datos simultáneamente.
- *register()*: vincula un flujo específico (en este caso el puerto serial representado por `sys.stdin`) al objeto de sondeo, indicándole que debe estar alerta ante eventos de lectura (`uselect.POLLIN`).

Una vez configurado el objeto de sondeo, la siguiente línea de código dentro del bucle `while True`:

```
if spoll.poll(10):
```

permite verificar si existen datos esperando ser procesados sin detener el avance del programa. Su funcionamiento es el siguiente:

- `poll(10)`: evalúa el búfer de entrada. El argumento numérico (`10`) representa el tiempo máximo de espera o *timeout* en milisegundos.
- Si se recibe un comando en ese lapso de tiempo, la condición se cumple y se procede a leer con `readline()`.
- Si transcurren los 10 milisegundos y no ha llegado nada por el puerto serial, la función devuelve una lista vacía, ignorando el bloque `if` y permitiendo que la Pico continúe ejecutando inmediatamente las siguientes líneas de código (por ejemplo, el encendido de otros actuadores o la instrucción `time.sleep(0.1)`).

Para comprobar las ventajas de este sistema no bloqueante, enviaremos comandos desde la terminal del computador y observaremos cómo la Raspberry Pi Pico es capaz de reaccionar de forma inmediata a la vez que sigue ejecutando su lazo principal de forma continua.

4 Desarrollo 3

4.1 Introducción a Interfaces Gráficas con Tkinter

Hasta este punto del laboratorio, la interacción con los programas se ha realizado exclusivamente a través de la terminal o consola de comandos. Para llevar nuestros proyectos de robótica a un nivel profesional, es fundamental desarrollar **Interfaces Gráficas de Usuario (GUI)**, las cuales permiten a un operador controlar sistemas físicos (como la Raspberry Pi Pico) mediante botones, indicadores y gráficos visuales.

Para este propósito utilizaremos **Tkinter**, la biblioteca estándar de Python para el desarrollo de GUIs. A diferencia de la programación secuencial tradicional, Tkinter utiliza un paradigma **orientado a eventos**, donde el programa no avanza línea por línea esperando, sino que se queda `escuchando` interacciones del usuario (como clics o pulsaciones de teclado) para reaccionar.

El código base para crear nuestra primera interfaz de control es el siguiente:

Codigo_Python/Ejemplo_4.py

A continuación, revisaremos los pilares fundamentales que componen cualquier aplicación gráfica en Tkinter basándonos en el código anterior.

Creación de la Ventana y el Bucle Principal: Todo programa con interfaz requiere un "lienzo" base donde se ubicarán los elementos, y un ciclo infinito que mantenga la ventana abierta y respondiendo.

Las instrucciones encargadas de esto son:

```
ventana = tk.Tk()
# ... (configuración de la ventana) ...
ventana.mainloop()
```

- `tk.Tk()`: inicializa la ventana principal del sistema operativo. A este objeto le podemos modificar propiedades como su título (`title()`) y dimensiones (`geometry()`).
- `mainloop()`: es el motor de la interfaz. Este método crea un ciclo infinito nativo que captura los eventos del sistema (clics, movimiento del mouse) y redibuja la pantalla. **Importante:** Cualquier código escrito después de `mainloop()` no se ejecutará hasta que el usuario cierre la ventana.

Widgets y Posicionamiento: Los elementos visuales (textos, botones, cuadros de entrada) se conocen como *Widgets*. En este ejemplo utilizamos `Label` y `Button`.

```
etiqueta_titulo = tk.Label(ventana, text="Laboratorio de Interfaces")
etiqueta_titulo.pack(pady=15)
```

- **Creación del Widget:** Al instanciar un widget (como `tk.Label`), el primer argumento siempre debe ser su elemento `padre` o contenedor (en este caso, `ventana`).
- **Gestor de Geometría (`pack`):** Un widget creado en memoria no será visible hasta que se le asigne una posición. El método `pack()` empaqueta los elementos uno debajo del otro automáticamente. El parámetro `pady` agrega un margen vertical en píxeles para mejorar la estética.

Interactividad y Callbacks (`command`): Para que un botón ejecute una acción real, debe estar enlazado a una función lógica, conocida en este contexto como *Callback*.

```
def accion_boton():
    etiqueta_estado.config(text="¡Botón presionado!", fg="green")

boton_prueba = tk.Button(ventana, text="Haz clic aquí", command=accion_boton)
```

- **Parámetro `command`:** Vincula el evento de `hacer click` con la función `accion_boton`. **Nota crucial:** Se debe pasar el nombre de la función *sin paréntesis* (`command=accion_boton`). Si se incluyen los paréntesis, la función se ejecutará inmediatamente al iniciar el programa y no al presionar el botón.
- **Actualización Dinámica (`config`):** Dentro del *callback*, utilizamos el método `.config()` sobre un widget existente (`etiqueta_estado`) para modificar sus propiedades en tiempo real (como su texto y color de fuente `fg`), logrando así una interfaz reactiva.

Al ejecutar este script, observaremos una ventana funcional. Al hacer clic en el botón, el entorno orientado a eventos detectará la interacción, ejecutará la función asociada y actualizará la etiqueta visual de forma instantánea, sentando las bases para enviar comandos por serial a nuestra Raspberry Pi Pico.

5 Desarrollo 4

5.1 Transmisión Serial desde la Interfaz Gráfica

Una vez comprendido el funcionamiento básico de los *widgets* y los eventos en Tkinter, el siguiente paso lógico es fusionar esta interfaz con la librería `pyserial`. En este ejercicio práctico, estableceremos la conexión con la Raspberry Pi Pico y configuraremos un único botón visual para enviar el comando de encendido a nuestro microcontrolador.

El código que integra ambos sistemas en el computador es el siguiente:

Codigo_Python/Ejemplo_5.py

A continuación, analizaremos los mecanismos clave que permiten a nuestra ventana de Tkinter comunicarse de manera estable con el hardware externo.

Inicialización Global y Gestión del Puerto Serial: A diferencia de los scripts secuenciales, en una interfaz gráfica el puerto serial debe declararse al inicio del programa. Esto permite que el puerto (`puerto_serial`) actúe como una variable global, manteniéndose abierto y accesible para cualquier botón de la interfaz mientras la ventana exista.

Para evitar que el puerto quede bloqueado en el sistema operativo al cerrar el programa, utilizamos el siguiente interceptor de eventos:

```
ventana.protocol("WM_DELETE_WINDOW", cerrar_aplicacion)
```

- `WM_DELETE_WINDOW`**:** Es un evento nativo del sistema operativo que se dispara cuando el usuario hace clic en la "Xroja para cerrar la ventana.
- `cerrar_aplicacion`**:** Es la función *callback* asignada, la cual se encarga de ejecutar explícitamente `puerto_serial.close()` para liberar el puerto COM antes de destruir la interfaz gráfica con `ventana.destroy()`.

Transmisión de Comandos desde un Evento Visual: La verdadera interacción ocurre cuando el usuario presiona el botón `Encender LED`. Este botón invoca a la función `enviar_encendido()`, la cual acondiciona y transmite el mensaje.

```
comando = "on\r\n"
puerto_serial.write(comando.encode())
```

- **Caracteres de Terminación (`\r\n`):** Como se estudió previamente, es obligatorio concatenar el retorno de carro y salto de línea para que la función `readline()` de la Raspberry Pi Pico detecte el final de la instrucción.
- **Codificación a Bytes (`encode()`):** El método `write()` del puerto serial no acepta texto plano (*strings*). La función `.encode()` transforma la cadena de texto en un flujo de *bytes* compatible con el estándar de transmisión UART.

Actividad Práctica

Desafío de Integración: Basándose en el código proporcionado, modifique el *script* de la interfaz gráfica para incorporar un segundo botón en la pantalla.

Este nuevo botón, que deberá tener un color distinto (por ejemplo, rojo o salmón), debe estar vinculado a una nueva función *callback* que ensamble y envíe el comando `'off\r\n'` mediante el puerto serial. Al finalizar, usted debe ser capaz de encender y apagar el LED de la placa de desarrollo haciendo clics en la interfaz de su computador.

El código con la solución lo pueden encontrar en:

Codigo_Python/desafio_ejemplo_5

6 Desafio Final

6.1 Implementación de Funciones Continuas: Comando Titilar

Hasta ahora, los comandos enviados a la Raspberry Pi Pico ejecutaban acciones instantáneas (encender o apagar). Sin embargo, en robótica es común requerir que un actuador realice una tarea repetitiva y continua, como hacer parpadear un LED a una frecuencia determinada, mientras el sistema sigue `escuchando` nuevas órdenes.

Si intentamos resolver esto utilizando la clásica función `time.sleep(0.5)`, bloquearemos el procesador durante ese medio segundo. Durante ese tiempo, la lectura del puerto serial (`spoll.poll()`) no se ejecutará, provocando que la placa ignore cualquier comando de apagado que enviemos desde la interfaz gráfica.

Para evitar este bloqueo, utilizamos una **Máquina de Estados** apoyada en el reloj interno del microcontrolador. A continuación, se le proporciona el bloque de código encargado de gestionar el parpadeo de forma asíncrona.

```
# EJECUCIÓN DEL PARPADEO (No bloqueante)
if estado_titilar:
    tiempo_actual = time.ticks_ms()
    if time.ticks_diff(tiempo_actual, ultimo_tiempo) >= 500:
        led.toggle() # Cambia de 1 a 0, o de 0 a 1
        ultimo_tiempo = tiempo_actual # Reinicia el cronómetro
```

Análisis del bloque proporcionado:

- **Variable de Estado (`estado_titilar`):** Todo el bloque está condicionado por esta variable booleana (*flag*). Si es `True`, el motor de parpadeo funciona. Si es `False`, se ignora por completo.
- **Temporización (`ticks_ms`):** Al usar `time.ticks_diff()`, el sistema evalúa constantemente si han pasado 500 ms desde la última vez que el LED cambió de estado. Si aún no pasa el tiempo, el procesador queda libre inmediatamente para volver a revisar el puerto serial.

Actividad Práctica Evaluada

Desafío: Lógica de Control de Estados

Haciendo uso del bloque de código proporcionado arriba, modifique su sistema para incorporar la función de parpadeo:

1. **En Python (PC):** Agregue un tercer botón a su interfaz gráfica llamado "Titilar LED". Este botón debe enviar el comando `"titilar\r\n"` por el puerto serial.
2. **En MicroPython (Pico):**
   - Declare las variables globales necesarias antes del `while True` (`estado_titilar = False` y `ultimo_tiempo = time.ticks_ms()`).
   - Modifique la etapa de **lectura serial** (dentro del `if spoll.poll(10):`) para capturar el nuevo comando `"titilar"` y activar la bandera correspondiente.
   - **Condición Crítica:** Asegúrese de que, si el sistema está parpadeando y se recibe un comando de `on` u `off`, la variable `estado_titilar` se desactive automáticamente para detener el ciclo continuo y ejecutar la nueva orden.

Si quiere implementar otro tipo de colores para sus botones en Tkinter, a continuación encontrara un link con todos los posibles colores en Tkinter:

Colores

Finalmente la solución de este desafio la puede encontrar a continuación:

- **Codigos_Python/desafio_final.py**
- **Codigos_RPI_Pico/desafio_final.py**

7 Referencias

1 Link 1

2 Link 2

3 Link 3